Telecom Network Monitoring Platform

Containerized Three-Tier Telecom Monitoring Dashboard

Team Mohammed Ahmed Ali
Mahmoud Ashraf Yadem
Ibrahim Samy

Date March 2026

Project Type Docker-based Telco Monitoring Platform

Instructor AZZA Khalil

This document describes the architecture, services, request flow, deployment, security posture, and operational model of the Telecom Network Monitoring Platfo

Contents

Containerized Three-Tier Telecom Monitoring Dashboard....1

1 Executive Summary.... 2

2 Project Objectives.... 2

3 High-Level Architecture 2
3.1 Architecture Notes....3

4 Service-by-Service Description 3
4.1 Nginx Reverse Proxy 3
4.2 Frontend Dashboard....4
4.3 Backend API 4
4.4 PostgreSQL Database 4
4.5 Data Updater 4

5 Request and Data Flow.... 4
5.1 Flow Explanation....5

6 Dashboard Overview 5

7 Docker Compose Design.... 6
7.1 Network Topology 6
7.2 Persistent Storage 6
7.3 Dependency Management 6

8 Detailed Port and Exposure Map 7

9 API Overview 7
9.1 Health and Status 7
9.2 Base Stations 7
9.3 Network Metrics 7
9.4 Alerts....7
9.5 Service Status....7

10 Database Model 7

11 Security Analysis 8
11.1 Current Security Measures....8
11.2 Recommended Improvements 8

12 Operational Runbook.... 8
12.1 Startup 8
12.2 Logs....8
12.3 Shutdown....8
12.4 Scaling Demonstration 9

13 Use in Team Objectives 9

14 Troubleshooting Guide 9

15 Conclusion.... 9

1 Executive Summary

The Telecom Network Monitoring Platform is a containerized three-tier application built to simulate and visualize telecom infrastructure monitoring in a modern DevOps environment. The platform exposes a browser-based dashboard through Nginx, serves application logic through a backend API, stores operational data in PostgreSQL, and continuously injects simulated telemetry through a dedicated Python data-updater service.

The project demonstrates secure service separation, internal-only database exposure, reverse-proxy routing, health-checked startup dependencies, and a dashboard suitable for demonstrating Docker Compose deployment, service isolation, and monitoring concepts in a telecom-focused scenario.

2 Project Objectives

The main objectives of the project are:

- Build a realistic telecom monitoring dashboard using a three-tier architecture.
- Demonstrate how Docker Compose can orchestrate multiple tightly-coupled services.
- Show secure networking practices by keeping the database internal to a dedicated network.
- Provide a platform for discussing image optimization, runtime hardening, Compose security, Vault integration, and private registry workflows.
- Simulate telecom base station metrics, alerts, and service availability for demonstration and testing.

3 High-Level Architecture

The deployed system consists of five main services:

1. **Nginx**: public entry point on port 80.
2. **Frontend**: Node.js/Express dashboard service.
3. **Backend**: Node.js/Express REST API.
4. **PostgreSQL**: internal data layer for metrics, alerts, and service state.
5. **Data Updater**: Python-based simulator that continuously writes telecom telemetry to the database.

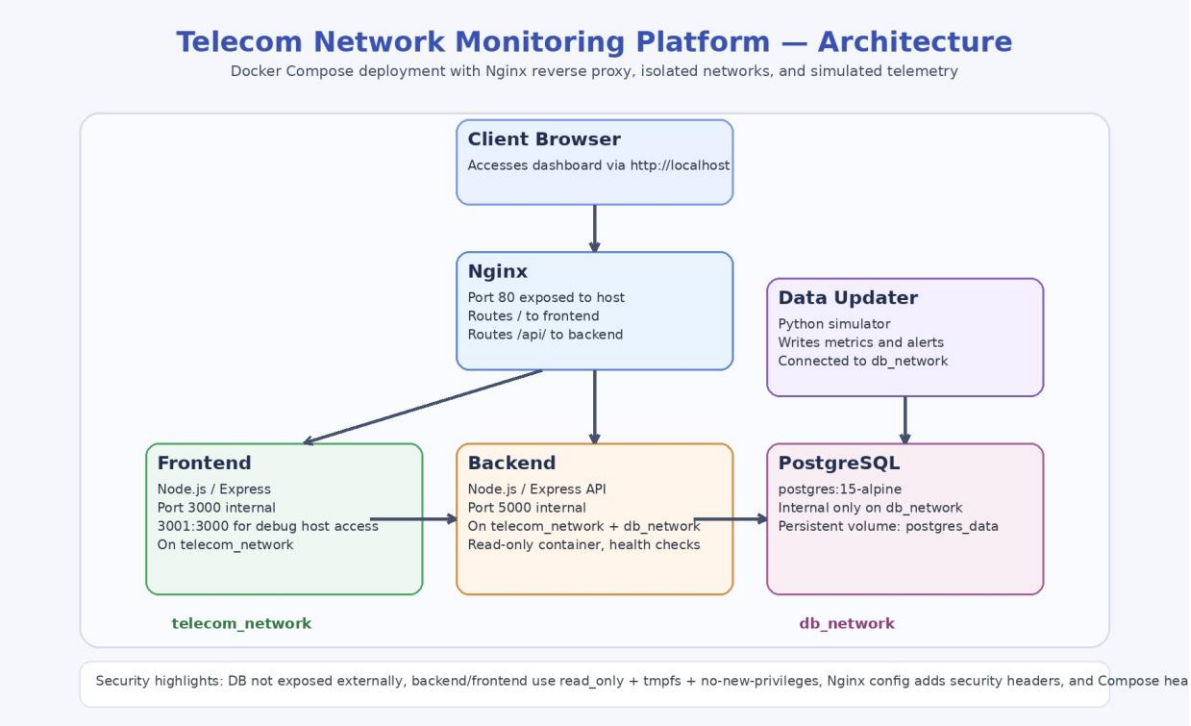

Figure 1: Architecture diagram generated from the supplied Docker Compose and Nginx configuration.

3.1 Architecture Notes

- Nginx is the only service published on host port 80 for primary user access.
- The frontend is also published on host port 3001 for direct debug access.
- The backend is not publicly published; it is only exposed internally.
- PostgreSQL is isolated from the host and reachable only on the dedicated database network.
- The backend joins both `telecom_network` and `db_network`, allowing it to bridge the application and data layers.

4 Service-by-Service Description

4.1 Nginx Reverse Proxy

Nginx acts as the browser-facing entry point. It receives incoming HTTP requests and forwards them according to path-based routing rules:

- `/` is forwarded to the frontend upstream.
- `/api/proxy/` is forwarded to the frontend because the frontend itself exposes proxy endpoints.
- `/api/` is forwarded directly to the backend upstream.

The configuration also enables several basic security headers, including `X-Content-Type-Options`, `X-Frame-Options`, and `Referrer-Policy`.

4.2 Frontend Dashboard

The frontend service is a Node.js/Express application that renders the telecom monitoring dashboard. It presents:

- base station status cards,
- network metric panels,
- active alert panels,
- and container-identification information useful for load-balancing demonstrations.

4.3 Backend API

The backend provides the application logic and REST endpoints used by the dashboard. It communicates with PostgreSQL and exposes service health and telecom monitoring data such as base stations, network metrics, alerts, and service status.

4.4 PostgreSQL Database

PostgreSQL stores the project data model, including base stations, network metrics, alerts, and service status records. It uses a persistent Docker volume and an initialization SQL script mounted into the container during startup.

4.5 Data Updater

The data-updater service is a Python-based simulator that continuously inserts telecom telemetry into the database, making the dashboard appear live and dynamic.

5 Request and Data Flow

The end-to-end request path is shown below.

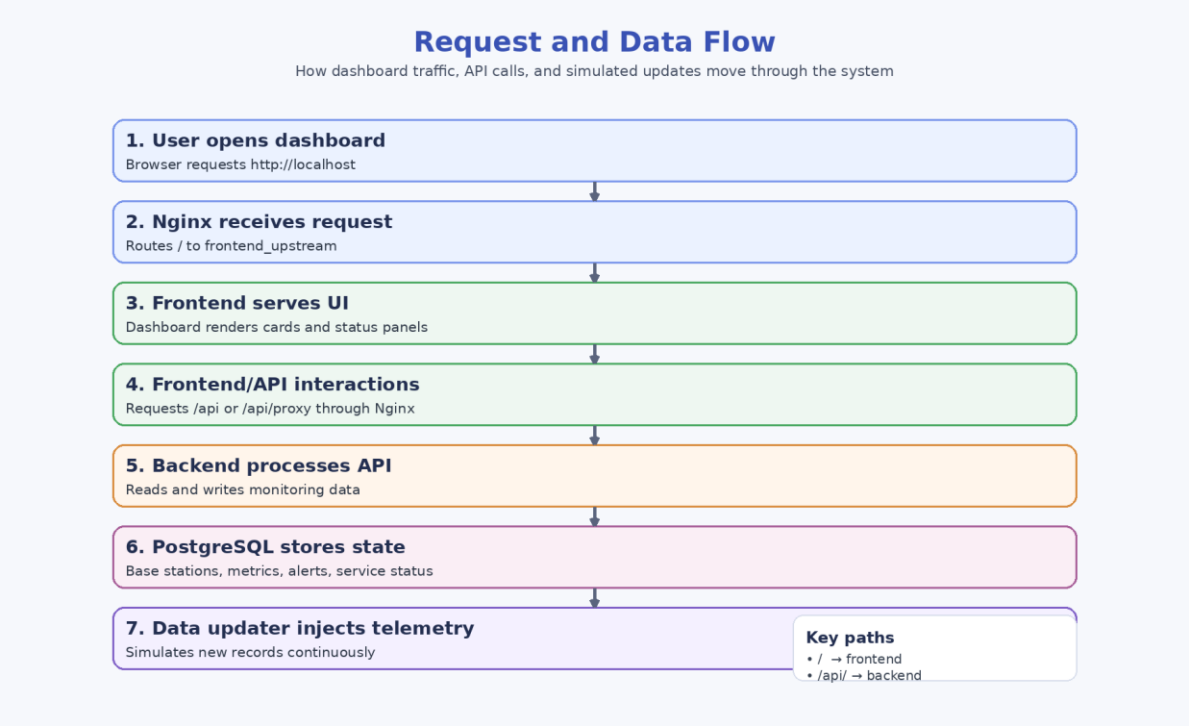

Figure 2: Request and data flow across client, reverse proxy, application services, and database.

5.1 Flow Explanation

1. A user opens the dashboard through http://localhost.
2. Nginx receives the request and routes the UI path to the frontend.
3. The frontend renders the dashboard and may request backend data through Nginx.
4. Requests under /api/ are forwarded to the backend.
5. The backend reads or writes records in PostgreSQL.
6. Independently, the data-updater service continuously inserts simulated metrics and alerts into the database.
7. The dashboard refreshes and displays near-real-time changes.

6 Dashboard Overview

The current dashboard presents a clean monitoring interface designed around three main panels: base station status, network metrics, and active alerts. It also shows which frontend and backend containers served the request, which is useful when demonstrating scaling or least-connection balancing.

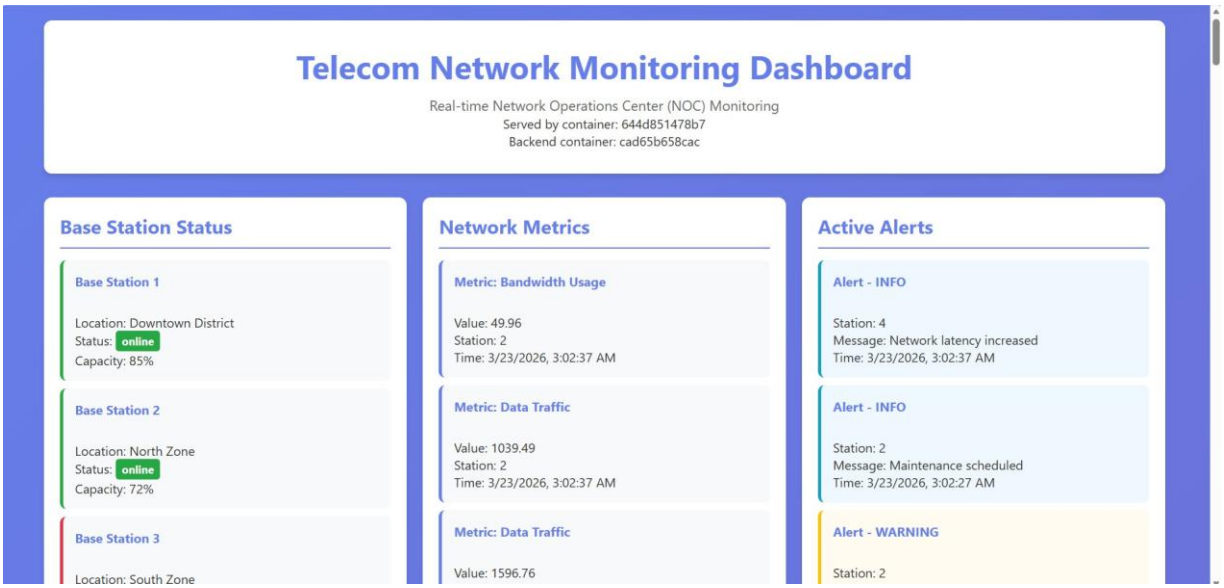

Figure 3: Provided screenshot of the Telecom Network Monitoring Dashboard.

7 Docker Compose Design

7.1 Network Topology

The Compose file defines two bridge networks:

- **telecom_network**: used by Nginx, frontend, and backend.
- **db_network**: used by PostgreSQL, backend, and the data-updater.

This split improves isolation by keeping the database layer outside direct access from Nginx and the browser.

7.2 Persistent Storage

A named volume, postgres_data, is used to persist PostgreSQL data across container restarts.

7.3 Dependency Management

The Compose file uses health-check-based dependency conditions so that:

- backend waits for PostgreSQL to become healthy,
- frontend waits for backend,
- Nginx waits for frontend and backend,
- data-updater waits for PostgreSQL.

8 Detailed Port and Exposure Map

Table 1: Ports, exposure model, and access scope

Service	Internal Port	Host Port	Access Scope
Nginx	80	80	Primary browser entry point
Frontend	3000	3001	Direct debug access from host
Backend	5000	Not published	Internal only through Compose network
PostgreSQL	5432	Not published	Internal only on database network
Data Updater	–	–	No direct user access

9 API Overview

9.1 Health and Status

- GET /api/health

9.2 Base Stations

- GET /api/base-stations
- GET /api/base-stations/:id

9.3 Network Metrics

- GET /api/network-metrics

9.4 Alerts

- GET /api/alerts
- POST /api/alerts

9.5 Service Status

- GET /api/service-status

10 Database Model

The application documentation identifies four main logical tables:

- base_stations
- network_metrics
- alerts
- service_status

These tables support dashboard rendering, alert display, and service monitoring logic.

11 Security Analysis

11.1 Current Security Measures

The supplied Docker Compose and Nginx configuration already include several meaningful hardening controls:

- PostgreSQL is not published to the host.
- The backend and frontend are configured as `read_only: true`.
- Temporary writable paths are limited through `tmpfs`.
- `no-new-privileges:true` is enabled where applicable.
- Linux capabilities are dropped and only selectively added.
- Compose health checks reduce unstable startup behavior.
- Nginx adds baseline browser security headers.

11.2 Recommended Improvements

To further strengthen the platform:

- move secrets from environment variables to HashiCorp Vault,
- enforce non-root users in all custom images if not already done in the Dockerfiles,
- add image scanning in CI using tools such as Trivy or Snyk,
- sign images and distribute them through a private registry,
- consider rate limiting and TLS termination on Nginx for production deployment.

12 Operational Runbook

12.1 Startup

```
docker compose up -d
docker compose ps
curl http://localhost/api/health
```

12.2 Logs

```
docker compose logs -f
docker compose logs frontend
docker compose logs backend
docker compose logs nginx
docker compose logs postgres
```

12.3 Shutdown

```
docker compose down
docker compose down -v
```

12.4 Scaling Demonstration

```
docker compose up -d --scale frontend=3
docker compose up -d --scale backend=3
```

13 Use in Team Objectives

This project is well suited for splitting work across five team objectives:

1. image size optimization,
2. image security hardening,
3. Compose security,
4. Vault-based secret management,
5. secure image distribution through a local or private registry.

14 Troubleshooting Guide

Table 2: Common troubleshooting cases

Symptom	Likely Cause	Suggested Action
Dashboard not reachable	**Nginx or frontend unhealthy**	Check docker compose ps and inspect Nginx/frontend logs
API returns errors	**Backend cannot reach database or backend unhealthy**	Check backend logs and PostgreSQL health
No live metrics appear	**Data-updater not running or database write issue**	Verify data-updater container logs and database connectivity
Database startup fails	**Initialization, credentials, or volume issue**	Review PostgreSQL logs and mounted init script
Load balancing demo not visible	**Only one replica running**	Scale frontend or backend and refresh dashboard repeatedly

15 Conclusion

The Telecom Network Monitoring Platform is a strong demonstration project for Docker, DevOps, and telecom-cloud learning. It combines a realistic user-facing dashboard with secure service isolation, reverse-proxy routing, health-checked orchestration, and simulated telemetry generation. As a result, it serves both as a technical learning platform and as a presentable portfolio project for interviews, reports, and academic submissions.

A Key Nginx Routing Summary

- `/` → frontend upstream
- `/api/proxy/` → frontend upstream
- `/api/` → backend upstream